

MPdetector: A Multi-Party Collaborative Federated Transfer Learning Approach for IoT Intrusion Detection

Li Lin , Member, IEEE, and ZhenKun Chen 

Abstract—The pervasive adoption of the Internet of Things (IoT) is accompanied by numerous network security threats, making the timely detection of anomalies in traffic data through intrusion detection increasingly critical. The existing intrusion detection methods based on federated learning can achieve good results under the condition of sufficient labeled data. However, the traffic data of participants in real IoT environments have various characteristics and there are a large amount of unlabeled data, which easily leads to the performance degradation of the intrusion detection model. To address this challenge, this paper proposes a novel federated transfer learning approach for IoT intrusion detection based on multi-party collaboration called MPdetector. MPdetector uses an encoder to extract the feature representation of diverse traffic data from heterogeneous clients, and maps the feature representation of each client to a unified feature space. In addition, a label transfer strategy is introduced to make full use of unlabeled data, and a new mapping function is used to reconstruct the traffic data of each client to expand client's local data set, which can further improve the detection performance of the model in varied and complex IoT environments. Theoretical analysis proves that the entire transfer learning process of MPdetector is conducted within a secure context. Experiments on four widely used intrusion detection datasets show that MPdetector can detect known and unknown abnormal traffic more accurately than the existing three classical intrusion detection algorithms, and has strong generalization. Meanwhile, the detection effect of MPdetector will be further improved with the increase of the volume of labeled traffic.

Index Terms—IoT, intrusion detection, federated transfer learning, deep learning.

I. INTRODUCTION

WITH the rapid development and widespread application of IoT technology [1], [2], [3], the number and types of IoT devices have increased rapidly in the past few years [4]. However, most IoT devices are small, low-cost, lack fixed power supplies, and are difficult to maintain. These characteristics often result in inadequate security considerations, making these devices more vulnerable to cyber-attacks [5] such as Zero-day

attacks and Distributed Denial of Service (DDoS) attacks [6]. According to statistics, IoT devices on the Internet suffered up to 5.7 billion attacks in 2020, most of which were DDoS and botnet attacks launched by malware such as Mirai and Gafgyt [7]. In addition, there are differences in terminal devices, applications and network protocols in IoT network, which makes network traffic highly heterogeneous and leads to increasingly complex patterns of network attacks. Consequently, detecting potential intrusions in IoT environment to avoid more threats is crucial for the healthy and sustainable development of IoT.

Traditional network intrusion detection methods are typically divided into two categories: signature-based and anomaly-based [8]. Signature-based methods identify intrusions by matching known attack features or patterns stored in a feature library. However, it is difficult for these methods to detect unknown attacks such as Zero-day attacks [9]. With the advancement of machine learning and deep learning, anomaly-based detection methods have shown improved performance in recognizing complex attack patterns and detecting Zero-day attacks [10]. Despite these advances, the training of these models often relies on single-source datasets i.e., traffic data collected from a single node. Single-source datasets are insufficient to reflect the complex network intrusions behaviors such as DDoS and botnet attacks that widely appear in IoT environments. For instance, botnet attacks typically involve large amounts of data from multiple nodes [11]. If the detection model lacks sufficient information, it often causes problems such as model overfitting and poor generalization.

In addition, the above schemes also ignore the privacy leakage problem introduced by collecting the data of IoT devices [12]. A common approach to address the privacy issue is the deployment of federated learning technology in network intrusion detection [13], where participant nodes only upload model parameters without sharing their local training data, a intrusion detection center server is responsible for aggregating the model parameters and distributing to the participant nodes, and the participant nodes update the model locally. However, most existing federated learning-based network intrusion detection schemes usually require data samples to have the same feature space. Then the heterogeneous traffic data in the real IoT environments will significantly affect the global model performance of federated learning IoT intrusion detection [14].

Several studies have proposed to use federated transfer learning technology to achieve knowledge sharing required for IoT

Received 2 January 2025; revised 10 July 2025; accepted 13 August 2025. Date of publication 19 August 2025; date of current version 3 December 2025. This work was supported by the China National Science Foundation under Grant 61502017. Recommended for acceptance by F. Restuccia. (Corresponding author: Li Lin.)

Li Lin is with the College of Computer Science, Beijing University of Technology, Beijing 100124, China, and also with the Beijing Key Laboratory of Trusted Computing, Beijing 100124, China (e-mail: linli_2009@bjut.edu.cn).

ZhenKun Chen is with the College of Computer Science, Beijing University of Technology, Beijing 100124, China (e-mail: 1317889183@qq.com).

Digital Object Identifier 10.1109/TMC.2025.3600306

intrusion detection while protecting participant's privacy [15], that is, leveraging rich labels in fully labeled data source domains to help build flexible and efficient detection models in target domains with large amounts of unlabeled data. Unlike traditional federated learning methods, federated transfer learning technology does not require participants to have the same features or the same sample space, so it is now receiving more and more attention. According to the number of participants, existing schemes can be divided into two categories. One is designed for two parties (a single source domain and a single target domain) [16], [17], which still faces the limitation of single-source data. The other is suitable for multiple parties [8], [18], [19], [20], [21], [22], but it requires the central server to have a common data set for initializing the model and the local training data set of participants must be labeled data. However, in real IoT environments, most traffic data is unlabeled. On the one hand, manual labeling requires professional knowledge and is time-consuming. On the other hand, labeled data usually raises privacy concerns, so many participating organizations or institutions only provide unlabeled data.

To address the above issues, exploiting the idea of SHFTL [23], this paper proposes a novel federated transfer learning approach for IoT intrusion detection based on multi-party collaboration called MPdetector. In MPdetector, each participating client extracts the feature representation of its traffic data, which is mapped into a unified feature space by a task manager to realize the alignment of feature representations of traffic data and reduce the difference between feature representations. A label transfer scheme is introduced to assign pseudo-labels to unlabeled network traffic data, and these pseudo-labeled data are integrated into the set of labeled data to participate in supervised training. Additionally, a deep learning-based traffic data reconstruction mechanism is proposed to facilitate feature transfer among clients, and a new mapping function is used to reconstruct unlabeled traffic data of each client, so as to fully integrate multi-source data information.

Compared with existing work, our contributions are summarized as follows.

- A novel IoT intrusion detection method is proposed, which uses multi-party semi-supervised federated transfer learning to better exploit unlabeled valuable data, and integrates feature representation alignment and reconstruction mechanisms, to solve the performance degradation problem of detection model caused by a large number of unlabeled data of participants and heterogeneity of data samples in existing IoT attack detection methods.
- A traffic data reconstruction mechanism is proposed to generate pseudo-labels from unlabeled data, which optimizes the mapping function in the classical federated transfer learning, achieves high detection accuracy without increasing the training cost and with minimal labeled data requirements, enhancing the ability of the model to identify malicious traffic data.
- The auxiliary training information provided by the unlabeled data in the target domain of intrusion detection was fully used to realize the collaborative training between a single source domain and multiple target domains, and the

generalization performance and detection performance of the model on target domains were improved.

- We have successfully implemented and evaluated MPdetector on the UNSW-NB15 [24], NSL-KDD [25], CICIDS2017 [26], and N-BaIoT [27], [28] datasets. The results demonstrate that MPdetector can achieve a detection accuracy of up to 98.2%, substantially outperforming the existing methods based on deep learning, semi-supervised learning, federated learning and semi-supervised federated learning.

II. PROBLEM STATEMENT

In the traditional IoT intrusion detection scenarios using machine learning or deep learning, IoT devices provide their local data as participants to participate in the network intrusion detection model training, and the final model is used to distinguish normal and abnormal traffic. However, these methods face two main challenges. On the one hand, the detection accuracy of these methods is limited by the small amount of labeled data per client. On the other hand, it is easy to expose the user privacy of IoT devices when aggregating network traffic data from multiple clients. Although the IoT intrusion detection scheme based on federated learning does not require clients to provide local traffic data, the model performance is still affected by the heterogeneity of network traffic data. To overcome the above issues, federated transfer learning can be employed to assign pseudo-labels to unlabeled data and then clients can use both labeled and unlabeled data for intrusion detection model training. Meanwhile each IoT client does not expose data information during the process of collaborative transfer learning, which ensures the security of users' privacy.

This paper proposes a method designed for scenarios involving multiple participants, where most of the data on the client side is unlabeled, as illustrated in Fig. 1. It is important to note that, unlike existing two-party-based approaches [17], the proposed method fully supports collaborative participation of over 100 clients in the same task. A central task manager coordinates with all clients to jointly perform federated transfer learning. This design offers greater generality and scalability. Assume that there are n IoT device participants, denoted as client k ($k = 1, 2, \dots, n$), participating in the collaboration. Each client possesses a network traffic dataset, represented as $X^1, X^2, \dots, X^n$, with the corresponding sample sizes $M^1, M^2, \dots, M^n$. Considering the heterogeneity of traffic data features between multiple clients, that is, there is no overlap of traffic data features between clients but their traffic data samples partially overlap. Denote the overlap of traffic data samples between the k th client and other clients as $O^k = \{x_i^k\}_{i=1}^{M_O^k}$ and the non-overlapping part as $N^k = \{x_j^k\}_{j=1}^{M_N^k}$, where M_O^k represents the number of overlapping samples and M_N^k represents the number of non-overlapping samples. Obviously there is $M^k = M_O^k + M_N^k$. Moreover, the overlapping traffic data is labeled as $y_o = \{y_i^k\}_{i=1}^{M_O^k}$, and the label information is stored by the task manager, while the non-overlapping traffic data remains unlabeled. A specific example of network traffic data distribution is illustrated in Fig. 2.

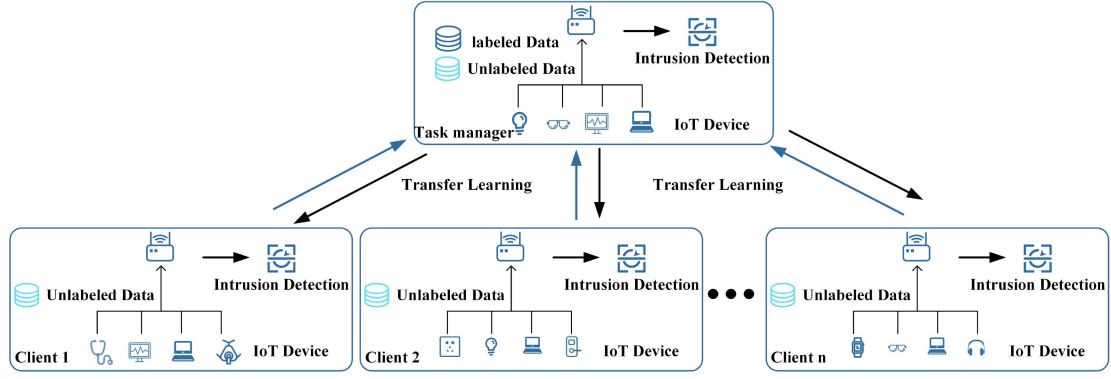


Fig. 1. IoT intrusion detection scenario with federated transfer learning.

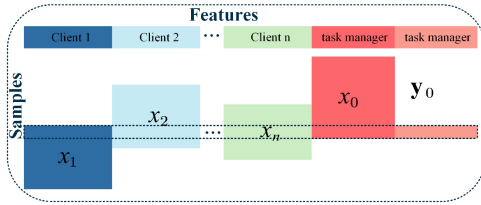


Fig. 2. Illustration of data distribution.

The goal of this paper is to propose a new federated transfer learning approach that is applicable to the above multi-party scenarios and has high performance. That is, the following problem will be solved.

The Optimal Federated Transfer Learning Problem is as follows.

Input: The number of participants n , the initial Intrusion Detection Classification Model g_{cla}^k , the local dataset of participants is X^k .

Question: Is there an optimal federated transfer learning approach that maximizes the ACC of intrusion detection classification models.

To address unlabeled traffic data, an effective feature extraction scheme and a label transfer scheme are presented in the proposed transfer learning approach. Participating clients provide unlabeled traffic data and extract network traffic feature representations. The task manager aligns these feature representations, assigns pseudo-labels to the unlabeled data, and reconstructs participating clients' traffic data using mapping functions.

To standardize the notation, the symbols used are defined in Table I.

III. DESIGN OF MPDETECTOR

In this section, we will give the design of the proposed approach MPdetector, including its working principles and detailed functions.

A. Sections and Subsections

As depicted in Fig. 3, in MPdetector, each participating client is responsible for achieving the feature representation extraction, while the task manager stores the overlapping network traffic

TABLE I
SYMBOL DEFINITIONS

Symbol	Meaning
n	number of clients
x^k	network traffic dataset of client
y_o	labeling of overlapping segments in client network traffic data
M^k	a sample size of the client k 's network traffic data
O^k, N^k	overlapping and non-overlapping portions of the client k 's network traffic data
M_O^k, M_N^k	number of samples in overlapping and non-overlapping portions of the client k 's network traffic data
$g_{enc}^k, g_{cla}^k, g_{dec}^k$	encoder, classifier, and decoder of client k
$\theta_{enc}^k, \theta_{cla}^k, \theta_{dec}^k, \theta_{res}^k$	corresponding model parameters of client K 's encoder, classifier, decoder, and mapping function
R_O^k, R_N^k	feature representation of labeled and unlabeled traffic data for client k
$\hat{y}_i^k, \hat{y}_p^k$	predictive labeling and pseudo-labeling of unlabeled traffic data for client k
R_U	global feature representation
R_{UO}^k, R_{UN}^k	unified feature representation of labeled and unlabeled traffic data for client k
W^k, D^k, S^k, Z^k	K-nearest neighbor matrix, degree matrix, normalization matrix, and probability matrix for client k
X_E^k, Y_E^k	expanded dataset and labeling for client k
E_{enc}^k	number of neurons for client k 's encoder
g_{enc}^k	encoder
F, H	mapping function
λ, w	weight parameter
$\mathcal{L}$	loss function

label information and handles feature representation alignment, label transfer, and feature transfer. The workflow of MPdetector is as follows.

- ① Each participating client trains an encoder to extract the feature representation of its traffic data and sends it to the task manager.
- ② The task manager aligns the feature representations of the clients to reduce the differences between the feature representations of different clients.
- ③ The task manager assigns pseudo-labels to the non-overlapping traffic data using the aligned global representation, selects the non-overlapping traffic data with

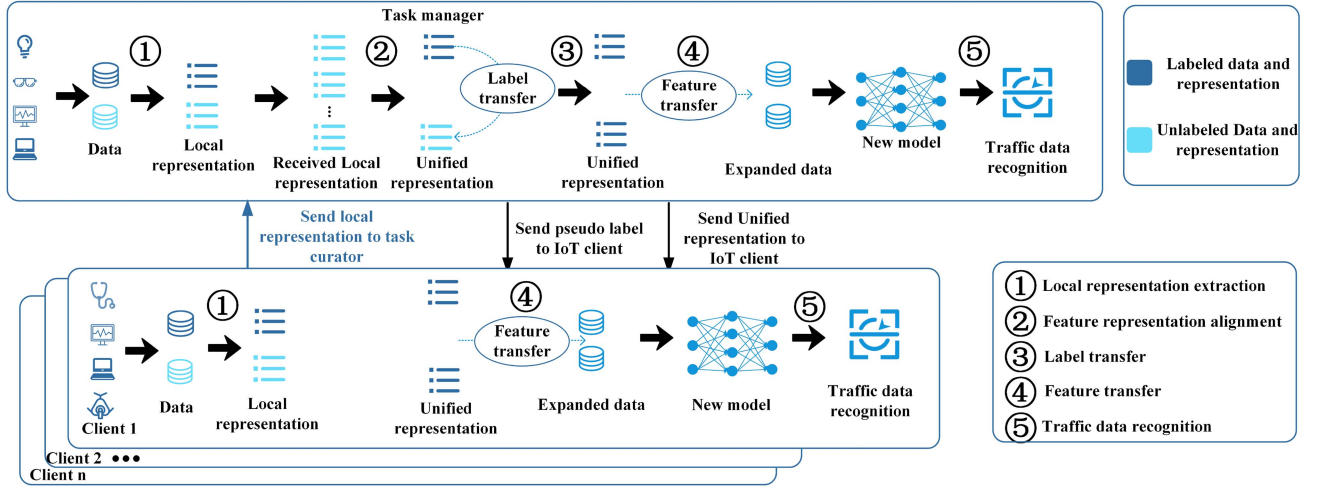


Fig. 3. Interaction flow between the task manager and the client.

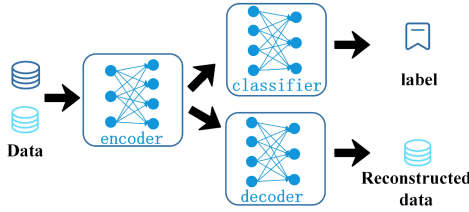


Fig. 4. Extracting feature representations of traffic data.

more reliable pseudo-labels to participate in the training of intrusion detection model, and sends the unified feature representation and pseudo-labels to the participating clients.

- ④ Each participating client uses an optimized traffic data mapping function to reconstruct the unified feature representation of non-overlapping traffic data from other clients, and adds the reconstructed traffic data to its training dataset.
- ⑤ Each participating client uses the extended data set to train and obtain an intrusion detection model, and then uses the model to classify and detect the traffic data.

B. Local Representation Extraction

In MPdetector, each client extracts the feature representation of its networking traffic data using a combination of encoder, classifier and decoder, and the extraction process is divided into two tasks, supervised classification and unsupervised reconstruction, according to whether the data has labels, as shown in Fig. 4.

For client k , as described in Table I, its overlapping traffic data O^k contains labeled data where labels information is stored by the task manager, its non-overlapping traffic data N^k contains unlabeled data. The encoder g_{enc}^k is used to extract feature representation $R_O^k = g_{enc}^k(O^k)$ of O^k and $R_N^k = g_{enc}^k(N^k)$ of N^k .

For the feature representation of labeled traffic data R_O^k , the task manager uses the classifier g_{cla}^k for classification. The

classification result is denoted as $\hat{y}_l^k = g_{cla}^k(R_O^k)$. The classification loss function is as follows.

$$\text{Arg min}_{\theta_{enc}^k, \theta_{cla}^k} \mathcal{L}_{cla}^k = \sum_{i=1}^{M_0^k} \ell_{XE}(\hat{y}_i^k, y_i^k), \quad (1)$$

where, $\theta_{enc}^k, \theta_{cla}^k$ represent the model parameters of the encoder g_{enc}^k and classifier g_{cla}^k , respectively, and ℓ_{MSE} is the cross-entropy loss function.

For the feature representation of unlabeled traffic data, the client uses the decoder g_{dec}^k for reconstruction. The reconstruction result is denoted as $\tilde{x}_j^k = g_{dec}^k(R_N^k)$. The reconstruction loss is defined as follows:

$$\text{Arg min}_{\theta_{enc}^k, \theta_{dec}^k} \mathcal{L}_{dec}^k = \sum_{j=1}^{M_N^k} \ell_{MSE}(\tilde{x}_j^k, x_j^k), \quad (2)$$

where θ_{dec}^k is the model parameter of the decoder g_{dec}^k and ℓ_{MSE} is the mean square error loss function. In this way, the final objective function for extracting the feature representation of traffic data is as follows.

$$\text{Arg min}_{\theta_{enc}^k, \theta_{cla}^k, \theta_{dec}^k} \mathcal{L}_r^k = \mathcal{L}_{dec}^k = \mathcal{L}_{cla}^k + \lambda \mathcal{L}_{dec}^k, \quad (3)$$

where $\lambda (\lambda \in (0, 1))$ is a hyperparameter that balances classification and reconstruction.

The specific feature extraction algorithm is outlined in Algorithm 1.

C. Feature Representation Alignment

In order to reduce the feature differences of traffic data between clients, MPdetector map the traffic data features of clients into a unified feature space, that is, to realize the alignment of traffic data features between clients.

First, the task manager uses client k 's classification accuracy w^k as the weight $w^k (\sum_{k=1}^n w^k = 1)$, to synthesize the feature representations of different clients' traffic data to obtain the global feature representation of traffic data $R_U (\sum_{i=1}^n w^i * R_O^i)$.

Algorithm 1: Local Feature Representation Extraction.

Input: Overlapping and non-overlapping portions of client k 's network traffic data $O^k = \{x_i^k, y_i^k\}_{i=1}^{M_O^k}$ and $N^k = \{x_j^k\}_{j=1}^{M_N^k}$, client k 's encoder g_{enc}^k , classifier g_{cla}^k , and decoder g_{dec}^k .

Output: The feature representation of client k 's labeled and unlabeled traffic data R_O^k and R_N^k , classification accuracy w^k of classifier g_{cla}^k .

- 1: **for** $k = 1$ to n **do**
- 2: Client k calculates $R_O^k = g_{enc}^k(x_i^k)$ and sends R_O^k to the task manager;
- 3: Client k calculates $R_N^k = g_{enc}^k(x_j^k)$;
- 4: $(\tilde{x}_j^k) = g_{dec}^k(R_N^k)$;
- 5: The task manager calculates $\hat{y}_l^k = g_{cla}^k(R_O^k)$;
- 6: The task manager calculates the classification loss $\mathcal{L}_{cla}^k$ according to Formula (1);
- 7: Client k calculates the reconstruction loss $\mathcal{L}_{dec}^k$ according to Formula (2);
- 8: Client k 's updates $\theta_{enc}^k, \theta_{cla}^k, \theta_{dec}^k$ based on $\mathcal{L}_{dec}^k$ and $\mathcal{L}_{cla}^k$;
- 9: Client k uses the trained g_{cla}^k to calculate w^k ;
- 10: **end for**
- 11: **return** R_O^k, R_N^k and w^k ;

The task manager uses the mapping function F^k to map the labeled traffic data feature representations and obtain the unified feature representation $R_{UO}^k = F^k(R_O^k)$. To ensure that the mapped feature representations retain their classification information, the classifier F_C^k is used to classify the mapped feature representations $\hat{y}_l^k = F_C^k(R_{UO}^k)$. Then, the objective function for traffic data feature mapping is then expressed as follows.

$$\arg \min_{\theta_{map}^k, \theta_{clb}^k} \mathcal{L}_U^k = \sum_{j=1}^{M_O^k} \ell_{MSE}(F^k(R_O^k), R_U) + \gamma \left(\sum_{i=1}^{M_{UO}^k} \ell_{XE}(F_C^k(R_{UO}^k), y_i) \right), \quad (4)$$

where θ_{map}^k and θ_{clb}^k are the model parameters of the mapping function F^k and the classifier $\hat{y}_l^k = F_C^k$, respectively $\gamma(\gamma \in (0, 1))$ is a hyperparameter. Using the optimized mapping function, the aligned uniform feature representation of labeled traffic data $R_{UO}^k = F^k(R_O^k)$ and the unified feature representation of unlabeled traffic data $R_{UN}^k = F^k(R_N^k)$ can be obtained.

The specific algorithm is shown in Algorithm 2.

D. Label Transfer

In real IoT environments, the amount of labeled traffic data available to each client is limited. Only using labeled traffic data to train intrusion detection models is easy to cause model overfitting. To fully utilize the information in unlabeled traffic data, a label transfer scheme is introduced into MPdetector, which enable the task manager to assign pseudo-labels to the

Algorithm 2: Feature Representation Alignment.

Input: The feature representation of the client k 's labeled and unlabeled traffic data, label set of the client k 's labeled traffic data $\{y_i^k\}_{i=1}^{M_O^k}$, classification accuracy w^k of classifier g_{cla}^k , and initialized mapping function F^k and classifier F_C^k ($k = 1, 2, \dots, n$).

Output: Unified feature representations of client k 's labeled and unlabeled traffic data R_{UO}^k and R_{UN}^k .

- 1: w^k ($k = 1, 2, \dots, n$) is normalized such that $(\sum_{k=1}^n w^k = 1)$;
- 2: $R_{UO}^k = F^k(R_O^k)$;
- 3: $R_{UN}^k = F^k(R_N^k)$;
- 4: calculate the reconstruction loss $\sum_{j=1}^{M_O^k} \mathcal{L}_{MSE}(R_{UO}^k, R_U)$;
- 5: $\hat{y}_l^k = F_C^k(R_{UO}^k)$;
- 6: calculates the classification loss $\sum_{i=1}^{M_O^k} \mathcal{L}_{XE}(\hat{y}_i^k, y_i^k)$;
- 7: update θ_{map}^k and θ_{clb}^k based on Formula (4);
- 8: **return** R_{UO}^k and R_{UN}^k ;

unlabeled traffic data by leveraging the similarity between the labeled and unlabeled traffic data. The label transfer process is illustrated in Fig. 5. The task manager concatenates feature representations R_{UO}^k and R_{UN}^k of the client k 's traffic data to obtain $R_U^k = [R_{UO}^k; R_{UN}^k]$, which is used to construct its W^k . A labeling matrix Y^k is defined for client k , where the row vectors in the Y^k corresponding to the one-hot encoded vectors of labeled traffic data and the rows for unlabeled traffic data are set to zero.

The probability matrix Z^k is used to assign pseudo-labels to unlabeled traffic data. The probability matrix Z^k can be obtained by using the conjugate gradient method to solve the linear system $(I - \alpha S^k)Z^k = Y^k$ [29], where I is the identity matrix, and $\alpha(\alpha \in (0, 1))$ is used to regulate the relative importance of a traffic data samples and its neighboring traffic data samples. Finally, based on the probability matrix Z^k , the pseudo-labels of unlabeled traffic data are calculated as follows.

$$\hat{y}_p^k = \text{Arg max}_j Z^k(i, j), \quad (5)$$

where the element $Z^k(i, j)$ represents the values at the i -th row and j -th column in the matrix Z^k .

The pseudo-labels derived from this process may not always be accurate, and training with incorrect pseudo-labels can degrade the performance of the client-side intrusion detection model. To mitigate this issue, the entropy value of traffic data samples is calculated to measure the uncertainty. The pseudo-labels with high certainty $\hat{y}_{ps}^k$ are selected to ensure accuracy, and the feature representation of corresponding traffic data samples N_h^k is expressed as R_{UNh}^k .

To further improve pseudo-label accuracy, each client initializes a NNR_w model. The model is trained using the global feature representations R_{UO}^k of labeled traffic data. After training completion, this model is applied to label unmarked traffic data, generating a membership matrix for each sample. This matrix represents the probability distribution of samples belonging to

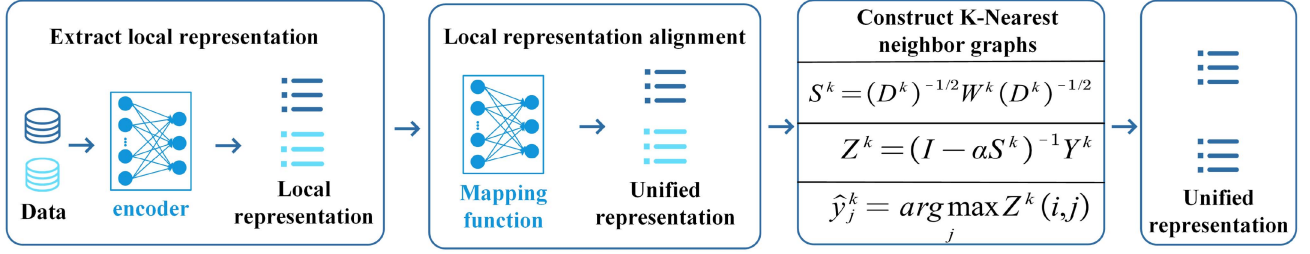


Fig. 5. K-nearest neighbors matrix propagation process.

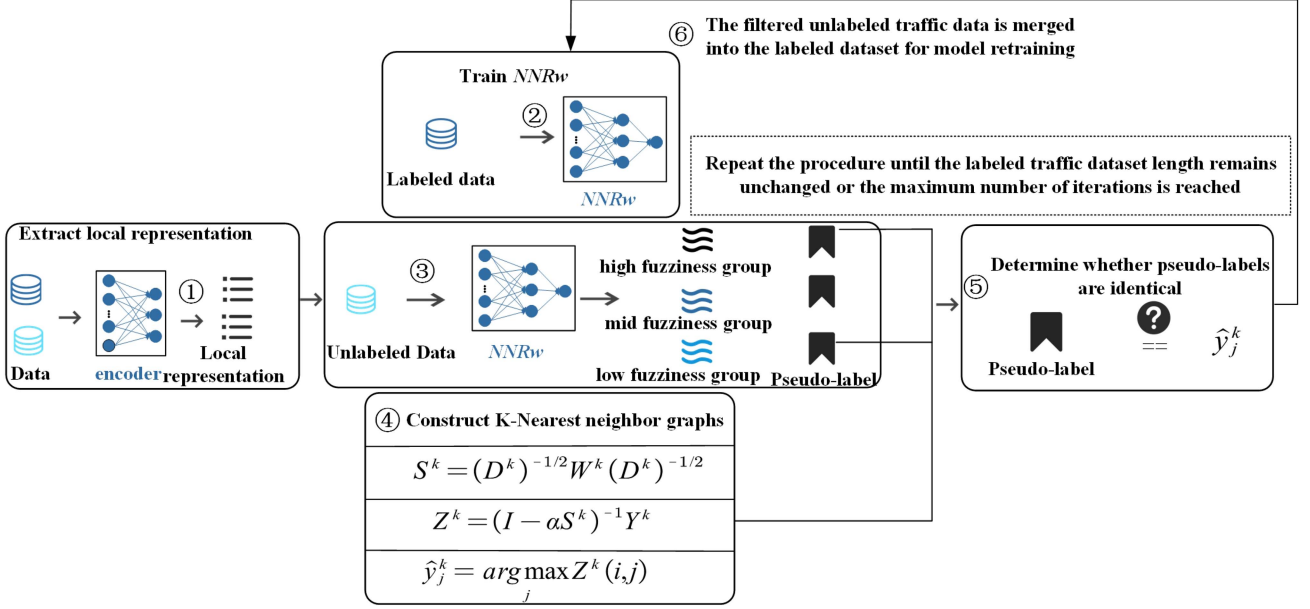


Fig. 6. Label transfer process.

different categories. The fuzziness of traffic data is then calculated through membership vectors $V = \{u_1, u_2, \dots, u_n\}$.

$$F(V) = -\frac{1}{n} \sum_{i=1}^n [u_i \log u_i + (1 - u_i) \log (1 - u_i)], \quad (6)$$

where u_1 represents the membership degree of the traffic data.

As shown in Fig. 6, the traffic data is divided into low fuzziness group FG_{low} , medium fuzziness group FG_{mid} , and high fuzziness group FG_{high} based on the fuzziness value $F(V)$. Samples belonging to FG_{mid} and FG_{high} are selected, and their labeling results from the NNR_w model are used as pseudo-labels $\hat{y}_{psf}^k$. Additionally, pseudo-labels $\hat{y}_{ps}^k$ for unlabeled traffic data are obtained using the K-nearest neighbor matrix label propagation method. The pseudo-labels generated by the NNR_w model are then compared with those obtained through the K-nearest neighbor matrix label propagation approach. Consistent results from both methods are selected as the final pseudo-labels for these unlabeled traffic data samples. These filtered unlabeled traffic data are subsequently merged into the labeled traffic dataset to retrain the NNR_w model. This process repeats until

either the size of the labeled traffic dataset stabilizes or the maximum number of iterations is reached. The detailed algorithm is provided in Algorithm 3.

E. Feature Transfer

Through the above label transfer scheme, client k can obtain pseudo-labels $\hat{y}_{ps}^k$ for unlabeled traffic data and the corresponding global feature representation R_{UNS}^k . However, since the feature information of each clients' traffic data is limited, a feature transfer scheme is introduced into MPdetector. In the scheme, a new nonlinear mapping function H^k is proposed, which is used to reconstruct the unlabeled traffic data from other clients and expand the missing features of the labeled traffic data by client k , so that the intrusion detection model can learn a more comprehensive network traffic feature representation and improve the ability of the model to identify abnormal traffic and normal traffic.

Compared with the linear mapping matrix, the deep learning model uses nonlinear activation functions to extract and learn the abstract features and representations of the data step by step through the multi-level structure [10]. It can better fit the network

Algorithm 3: Label Transfer.

Input: global feature representations R_{UO}^k and R_{UN}^k for client k 's labeled and unlabeled traffic data ($k = 1, 2, \dots, n$).

Output: highly accurate pseudo-labels $\hat{y}_{ps}^k$ and the corresponding global feature representation R_{UNS}^k .

- 1: **for** $k = 1$ to n **do**
- 2: $R_U^k \leftarrow [R_{UO}^k; R_{UN}^k]$;
- 3: Generate W^k where $W_{ij}^k = \exp(-\frac{\|R_U^k(i,:) - R_U^k(j,:)\|^2}{\sigma^2})$ ($i \neq j$);
- 4: $S^k \leftarrow (D^k)^{-1/2} W^k (D^k)^{-1/2}$;
- 5: $Z^k \leftarrow (I - \alpha S^k)^{-1} Y^k$;
- 6: Calculate $\hat{y}_p^k$ using Formula (5);
- 7: **while** iteration $\leq T$ **do**
- 8: Calculate $F(V)$ using Formula (6);
- 9: Select samples R_{UNE}^k from both FG_{mid} and FG_{high} groups;
- 10: Check consistency between $\hat{y}_{psf}^k$ and $\hat{y}_p^k$, then send matching pseudo-labels $\hat{y}_{ps}^k$ to clients;
- 11: $\hat{y}_{psf}^k = NNR_w(R_{UNE}^k)$;
- 12: Select high confidence $\hat{y}_{ps}^k$ and denote the corresponding feature representation as R_{UNS}^k ;
- 13: **end while**
- 14: Send $\hat{y}_{ps}^k$ and R_{UNS}^k to client k ;
- 15: **end for**
- 16: **return** $\hat{y}_{ps}^k$ and R_{UNS}^k ;

traffic data, understand the internal structure of traffic data more accurately, and then realize more effective data reconstruction. However, in real IoT environment, most of IoT devices have limited computing and storage resources, so it is more important to save the cost of client-side intrusion detection model training. Therefore, MPdetector uses a nonlinear deep learning model to reconstruct traffic data, which can not only reduce the pressure of client model training, but also improve the reconstruction effect of traffic data.

The objective function of traffic data feature reconstruction can be expressed as follows.

$$\text{Arg min}_{\theta_{res}^k} \mathcal{L}_C^k = \sum_{j=1}^{M_O^k} \ell_{MSE}(R_{UNS}^k, H^k(R_{UO}^k)), \quad (7)$$

where θ_{res}^k is the model parameter of the mapping function H^k .

Client k ($k = 1, 2, \dots, n$) uses the trained mapping function H^k to reconstruct the unlabeled traffic data from other clients $\tilde{x}_{js}^{kt} = H^k(x_i^t)$. Then, the reconstructed traffic data and the original traffic data set of client k can be merged to obtain the extended traffic data set $X_E^k = \{x_i^k, \tilde{x}_{js}^{k1}, \dots, x_{js}^k, \dots, \tilde{x}_{js}^{kn}\}$, with the corresponding label set $Y_E^k = \{y^k, \{\hat{y}_j^k\}_{j=1}^k\}$. Since the traffic data feature representation needs the original traffic data dimension to reconstruct the original traffic data, the original traffic data dimension is only accessed by client k , and other parties cannot obtain the information of the original traffic data only through

Algorithm 4: Feature Transfer.

Input: global feature representations R_{UO}^k and R_{UNS}^k of the clients' labeled and unlabeled traffic data, mapping function H^k ($k = 1, 2, \dots, n$).

Output: the expanded traffic dataset X_E^k of client k .

- 1: reconstruct the global feature representation of the labeled traffic data $H^k(R_{UO}^k)$;
- 2: calculate the reconstruction loss $\mathcal{L}_k$ based on Formula (6);
- 3: update θ_{res}^k according to the reconstructed loss $\mathcal{L}_k$;
- 4: $(\tilde{x}_{js}^{kt}) = H^n(x_i^t)$ is sent to other client;
- 5: merges the data set $X_E^k = \{x_i^k, \tilde{x}_{js}^{k1}, \dots, x_{js}^k, \dots, \tilde{x}_{js}^{kn}\}$;
- 6: **return** X_E^k ;

Algorithm 5: Traffic Data Recognition.

Input: client k 's trained encoder g_{enc}^k and classifier g_{cla}^k , and its network traffic dataset X^k .

Output: predicted labels $\hat{Y}^k$ for the network traffic dataset X^k .

- 1: $\hat{Y}^k = g_{cla}^k(g_{enc}^k(X^k))$;
- 2: **return** $\hat{Y}^k$;

the traffic data feature representation, which ensures the privacy security of client k . The detailed algorithm is shown as follows.

F. Traffic Data Recognition

Next, client k uses the expanded traffic dataset X_E^k to train an intrusion detection model to accurately identify normal traffic and abnormal traffic. To achieve this goal, client k retrains the encoder g_{enc}^k and classifier g_{cla}^k , to minimize the following objective function.

$$\text{Arg min}_{\theta_{enc}^k, \theta_{cla}^k} \mathcal{L}_{cla}^k = \ell_{XE}(g_{cla}^k(g_{enc}^k(X_E^k)), Y_E^K), \quad (8)$$

where θ_{enc}^k and θ_{cla}^k are the model parameters of encoder g_{enc}^k and classifier g_{cla}^k , respectively.

The detail is presented in Algorithm 5.

IV. THEORETICAL ANALYSIS**A. Complexity Analysis of Mapping Functions**

Theorem 1: Let H^k be the new mapping function for feature transfer on client side in MPdetector, where a deep neural network model replaces the mapping matrix $M^{(n*)}$. The computational complexity of H^k is $O(E_{iter} M_O^n \prod_{l=1}^{L-1} S_l S_{l+1})$, which is approximately the same as the computational complexity $O(M_O^{n*3})$ of $M^{(n*)}$.

Proof: The computational cost of matrix mapping mainly lies in matrix multiplication. The matrix $M^{n*} = R_{UNS}^n R_{UO}^n T (R_{UO}^n R_{UO}^n T + \mu I)^{-1}$ has a computational complexity of $O(M_O^{n*3})$, where M_O^n represents the number of overlapping traffic data samples.

For deep neural network mapping, the computational cost is concentrated in the forward and backward transfer phases.

If the network has L layers with S_l neurons in each layer, and the number of iterations during training is E_{iter} , the time complexity is $O(E_{iter}M_O^n \prod_{l=1}^{L-1} S_l S_{l+1})$, where, $E_{iter} = epochs * (M_O^n / batch_size)$. And the epochs and batch size are approximately the same, that means, $epochs \approx batch_size$, so $E_{iter} \approx M_O^n$. With the increase of the amount of traffic data M_O^n , in order to ensure that the deep learning model can better fit the traffic data structure, the number of layers of the neural network may increase, and the model can still converge when $\prod_{l=1}^{L-1} S_l S_{l+1} \approx M_O^n$. $\square$

Theorem 1 indicates that the computational complexity of deep neural networks is approximately the same as that of matrix mapping. This means that, compared with the traditional lightweight matrix mapping reconstruction methods applicable to resource-constrained IoT devices, the method in MPdetector, which uses deep learning (non-linear mapping functions) to reconstruct traffic features, does not introduce additional time overhead.

B. Security Analysis

Theorem 2: During the the feature representation alignment process in MPdetector, the task manager cannot retrieve client k 's original traffic data ($k \in (1, n)$).

Proof: If the task manager want to obtain client k 's traffic data O^k and N^k ($k \in (1, n)$), it must access the feature dimensions of O^k and N^k and client k 's encoder parameters θ_{enc}^k . As stated in Section III, the task manager can only access to the the encoded representations R_O^k and R_N^k in MPdetector but cannot get the feature dimensions of the traffic data. $\square$

Theorem 3: During the label transfer process in MPdetector, the task manager cannot retrieve client k 's original traffic data k ($k = 1, 2 \dots, n$).

Proof: During the label transfer process, if the task manager want to access client k 's original traffic data O^k and N^k , the number of the unified feature representations R_{UO}^k and R_{UN}^k must exceed the feature dimension. The labels corresponding to the unified feature representations R_{UO}^k and R_{UN}^k have two categories: $\hat{y}_{is}^k \in (0, 1)$ and $y_o \in (0, 1)$. The feature dimension of R_{UO}^k and R_{UN}^k is equal to the number of neurons E_{enc}^k in the final layer of the encoder g_{enc}^k , and since $E_{enc}^k \gg 2$, the feature dimensions of R_{UO}^k and R_{UN}^k will also $\gg 2$, therefore the task manager is prevented from accessing the client k 's original traffic data. the labels corresponding to the uniform feature representations R_{UO}^k and R_{UN}^k are $\hat{y}_{is}^k \in (0, 1)$ and $y_o \in (0, 1)$, representing two classes. The feature dimension of R_{UO}^k and R_{UN}^k is determined by the number of neurons E_{enc}^k in the final layer of the encoder g_{enc}^k . If $E_{enc}^k \gg 2$, the feature dimensions of R_{UO}^k and R_{UN}^k will also greatly exceed 2. Therefore, the task manager is prevented from accessing client k 's original traffic data. $\square$

Theorems 2 and 3 show that even if the task manager is untrusted, it cannot obtain the clients' original traffic data through feature representations or pseudo-labels of traffic data, thus ensuring the security of clients' data. That is, the entire federated transfer learning process in MPdetector is conducted within a secure context.

V. EXPERIMENT EVALUATION

In this section, comprehensive experiments have been conducted to evaluate the effect of MPdetector.

A. Experimental Environment

We have implemented MPdetector using Python3 and PyTorch to achieve deep learning algorithm. An autoencoder is used as the feature extraction model, a decoder is used to reconstruct the traffic data, and a classifier is used to classify the traffic data. The learning objective is to classify the traffic data into two classes. The learning rate is set to 0.0001, and the experiments is conducted on Intel Core i9-12900H, 2.50 GHz CPU, 16 GB memory.

Four well-known intrusion detection datasets: UNSW-NB15 [24], NSL-KDD [25], CICIDS2017 [26], and N-BaIoT [27], [28] are used to evaluate the performance of MPdetector. The UNSW-NB15 dataset was collected in 2015 from a real network environment at the Australian Security Laboratory. It includes a diverse range of network traffic, featuring both common network attacks and normal traffic. The NSL-KDD dataset is a refined version of the classic KDD dataset, created by removing duplicate samples from the original training and test sets. The CICIDS2017 dataset contains daily benign activity traffic data and various types of attacks, closely resembling real-world data. The N-BaIoT dataset consists of raw traffic captured from nine kinds of IoT devices infected by the Mirai and BASHLITE botnets.

The experimental procedure consists of the following steps. First, we preprocess the aforementioned datasets by randomly sampling the processed traffic data to create overlapping and non-overlapping traffic data for IoT clients. Without loss of generality, the non-overlapping data is then divided into training, validation, and test sets with a ratio of 7:2:1.

MPdetector fully supports applications in scenarios with homogeneous traffic data or partial feature overlap. However, to experimentally verify its detection performance under fully heterogeneous client traffic data, we partition the traffic features to simulate a feature-heterogeneous environment, where the feature spaces of different clients do not overlap at all, as shown in Table II. To simulate such a setting, in our experiments, if there are n clients participating in training, the traffic features are divided into n parts. Each client is then assigned a distinct feature space, achieving feature heterogeneity. However, to extend this partitioning to over 100 clients, the features would need to be split into more than 100 parts. Due to the limited feature dimensions in existing intrusion detection datasets, this would inevitably cause overlap between client feature spaces, making it impossible to simulate a completely heterogeneous scenario. Therefore, to evaluate MPdetector in a setting where feature spaces are entirely disjoint, we conducted experiments on multiple clients (without loss of generality, $n = 3$) using datasets with limited feature dimensions. Among these, the client responsible for maintaining network traffic data labels serves as the task manager, which typically initiates the intrusion detection model training process. The remaining clients function

TABLE II
FEATURE PARTITIONING OF THE DATASET

Datasets	Feature of client 1	Feature of client 2	Feature of client 3	Total features
UNSW-NB15	20	20	20	60
NSL-KDD	41	41	41	123
CICIDS2017	27	27	27	81
N-BaIoT	39	39	39	117

as participants. The final experimental results are reported as the average performance across all participating clients.

B. Classification Performance

In the experiment, MPdetector was compared with deep learning (DL), semi-supervised deep learning (SDL), federated learning (FL), and semi-supervised FL (SFL) based intrusion detection methods. In DL-based methods, each client uses an autoencoder to extract feature representations from the training data and then trains a classifier to detect intrusions. In SDL-based methods, each client extracts feature representations using an autoencoder, clusters the non-overlapping traffic data with K-means, and then trains classifiers on both overlapping and non-overlapping traffic data. The FL-based method applies the FedAVG algorithm to detect intrusions using an aggregated model. In SFL-based methods Each client employs an autoencoder to extract feature representations from training data. The K-means technique is then applied to cluster feature representations of non-overlapping traffic data. Both overlapping and non-overlapping traffic data are subsequently used as inputs for the FedAVG algorithm during federated training. Finally, the aggregated model obtained from this training process is utilized to evaluate the training data.

In the experiment, some common machine learning performance indicators are used to compare and evaluate the performance of different intrusion detection methods as follows.

Accuracy (ACC) is the ratio of traffic data correctly identified by the intrusion detection model, which is calculated according to the following Formula (8).

$$ACC = \frac{TP + TN}{TP + TN + FP + FN}, \quad (9)$$

False Alarm Rate (FAR) represents the proportion of normal traffic data being misclassified as abnormal traffic data resulting in false positives and is calculated according to Formula (9).

$$FAR = \frac{FP}{TN + FP}, \quad (10)$$

The F1 Score (F_1) provides a balanced evaluation of precision and recall. It is calculated using (10):

$$F_1 = \frac{TP}{TP + \frac{1}{2}(FP + FN)}, \quad (11)$$

In the above equations, TP represents correctly classified normal traffic data, FP refers to normal traffic data misclassified as abnormal, TN represents correctly classified abnormal traffic, and FN refers to abnormal traffic misclassified as normal.

TABLE III
COMPARISON ON ACC, FAR, AND F_1 FOR DIFFERENT DATASETS UNDER FIVE METHODS

Datasets		DL	SDL	FL	SFL	MPdetector
UNSW-NB15	ACC	95.56	95.95	93.22	93.76	98.30
	FAR	4.73	1.30	8.71	3.61	0.18
	F_1	94.95	95.19	92.55	92.67	98.12
NSL-KDD	ACC	94.84	92.57	94.96	95.54	97.77
	FAR	6.80	6.29	5.25	5.85	2.74
	F_1	94.22	91.53	94.24	94.98	97.85
CICIDS2017	ACC	93.76	91.21	93.03	93.11	98.88
	FAR	8.32	3.42	9.75	10.55	1.20
	F_1	93.06	89.23	92.34	94.84	98.60

As shown in Table III, FL performs worse than DL on the UNSW-NB15 and CICIDS2017 datasets. It is due to the different training data characteristics used by each client, and the heterogeneity of data feature space between clients, which affects the performance of FL. For the UNSW-NB15 dataset, SDL achieves higher detection accuracy than DL. However, on the NSL-KDD and CICIDS2017 datasets, SDL performs worse than DL. This is because when using K-means technology for clustering, some traffic data clustering results do not match the actual labels, which affects the performance of the final model. On relevant datasets, SFL consistently demonstrates higher accuracy than FL, as semi-supervised learning effectively addresses the insufficiency of labeled data across clients. However, the persistent feature heterogeneity among clients results in SFL's accuracy remaining lower than SDL's performance. MPdetector demonstrates superior performance across all evaluated datasets, achieving particularly outstanding results on the CICIDS2017 dataset with 98.86% accuracy. This exceptional performance stems from the fine-grained feature characteristics of CICIDS2017 dataset, combined with MPdetector's collaborative transfer mechanism that effectively integrates comprehensive feature representations. The model's ability to precisely capture complex attack patterns is significantly enhanced through this feature integration approach. Experimental results confirm that MPdetector exhibits strong generalization capabilities across diverse datasets. Compared with DL, SDL, FL, and SFL methods, MPdetector consistently achieves higher accuracy and F1 scores while maintaining lower false alarm rates.

In MPdetector, the task manager acts as the knowledge-rich source domain, while the clients act as the knowledge-poor

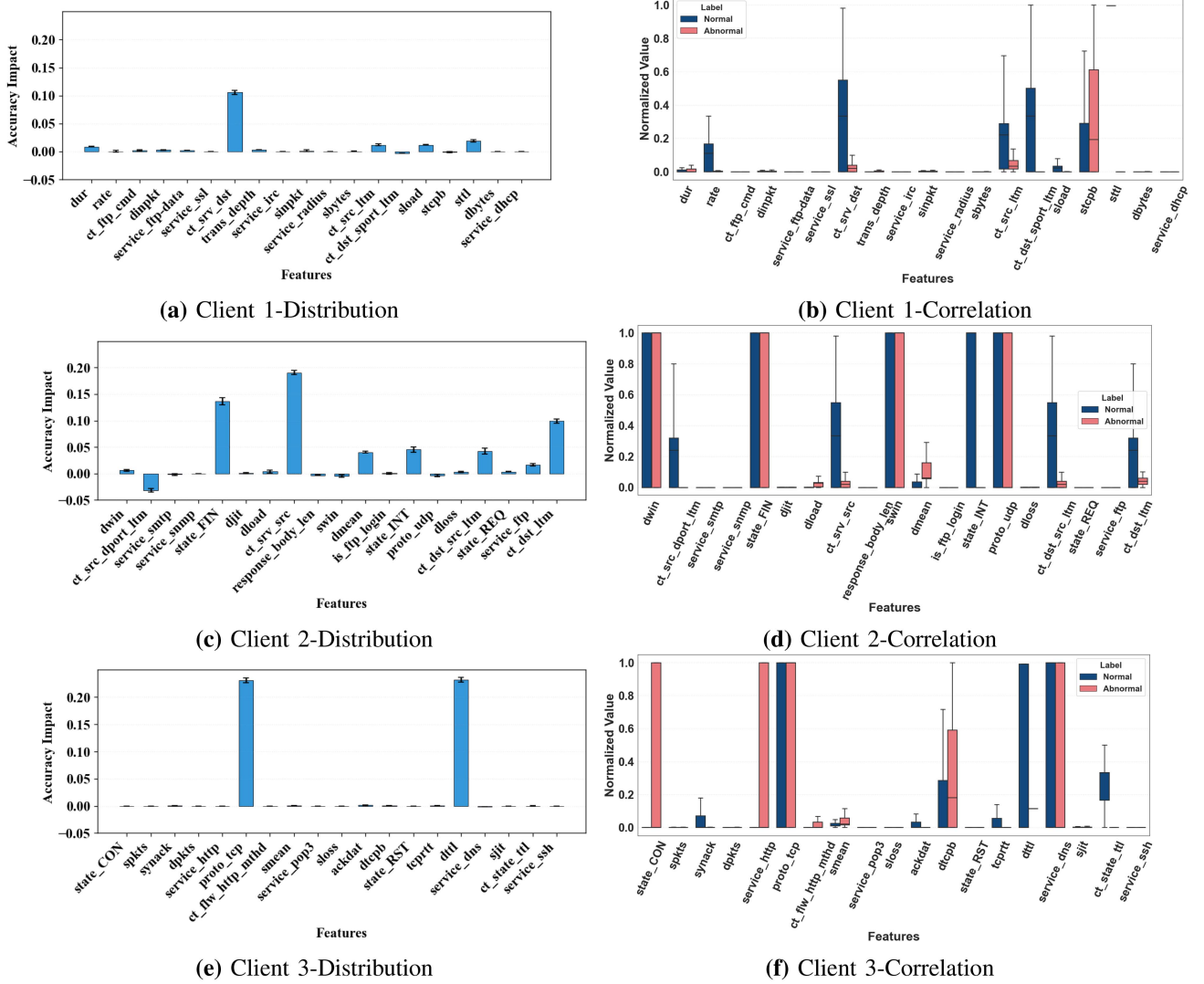


Fig. 7. Feature distribution and correlation analysis on the UNSW-NB15 dataset.

target domain. Collaborative knowledge transfer occurs between the task manager and multiple clients. This cooperative framework helps the clients train intrusion detection models, effectively addressing the issue of insufficient labeled data. Additionally, MPdetector employs a label transfer mechanism to assign pseudo labels to unlabeled data, and uses a nonlinear data reconstruction mechanism to achieve feature transfer to enrich the feature representation of traffic data. These measures collectively solve the problem of data heterogeneity and significantly improve the detection accuracy of client traffic analysis.

To better interpret the detection results, we conducted feature correlation analysis on the UNSW-NB15 dataset based on MPdetector's outputs, as illustrated in Fig. 7. The analysis reveals that connection timing features (ct_srv_dst , ct_src_lrm , ct_srv_src , ct_dst_lrm) and traffic behavior features (Sload, Sttl, Swin) show the most significant correlations with intrusion detection. Specifically, ct_srv_dst effectively identifies abnormal service access patterns by counting connection frequencies between specific services and destination addresses, while ct_src_lrm and ct_dst_lrm detect lateral movement behaviors

by tracking connection frequencies of source/destination addresses within time windows. Abnormal values in Sload (source instantaneous transmission rate) and Sttl (initial time-to-live) typically correlate with Exploits or Shellcode attacks, as these often involve non-standard packet rates or forged hop counts. Swin (TCP receive window size) reflects protocol-level flow control anomalies. Additionally, Proto (protocol type) and Dttl (destination-to-source TTL value) further assist in detecting abnormal behaviors, such as UDP protocol abuse or TTL manipulation. These features collectively form the detection basis for MPdetector, and the correlation analysis demonstrates its effectiveness in capturing attack-related anomalies.

Fig. 7 shows the feature distribution of traffic data, comparing normal and anomalous traffic across clients. The most distinguishable features between normal and anomalous traffic are crucial for model detection. For Client 1, significant distribution differences appear in ct_srv_dst , ct_src_lrm , and $ct_dst_sport_lrm$. The median ct_srv_dst value for normal traffic reaches 0.3, substantially higher than anomalous traffic, with a wider interquartile range. For Client 2, clear differences appear

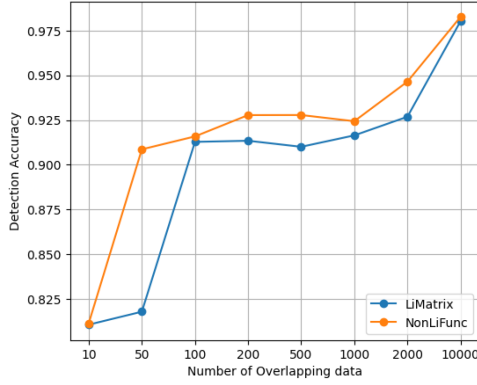


Fig. 8. Different numbers of overlapping data versus detection accuracy under two different traffic mapping functions.

in `ct_srv_src`, `ct_dst_src_item`, and `ct_dst_ltm`. Although `swin` and `proto_udp` demonstrate high correlation with detection results in Fig. 7, their distributions show no discernible difference between normal and anomalous traffic, potentially leading to model misclassification. For Client 3, marked differences exist in `dttl` and `ct_state_ttl` between traffic types.

C. Influence of Different Mapping Function

This experiment evaluates whether the new nonlinear mapping function proposed in the feature transfer part of MPdetector can improve the detection performance while ensuring the computational overhead of participating clients. We compare detection accuracy of two mapping functions: a linear mapping matrix (LiMatrix) and the nonlinear mapping function (NonLiFunc) in MPdetector, across various amounts of overlapping traffic data. Detection accuracy is the model performance trained on both overlapping and non-overlapping data. As illustrated in Fig. 8, the horizontal axis represents the number of overlapping data, and the vertical axis represents detection accuracy. The results show that NonLiFunc significantly improves the performance of the intrusion detection model. Notably, the enhancement is more pronounced when the number of overlapping data reaches 50 and 1,000 rows.

D. Influence of Different Amounts of Overlapping Data

This experiment examines how different amounts of overlapping data affect pseudo-labeling accuracy and detection accuracy, when MPdetector is applied. Pseudo-label accuracy is the accuracy of pseudo-labels assigned to non-overlapping data during label transfer process, detection accuracy is the model performance trained on both overlapping and non-overlapping data. The results are illustrated in Figs. 9 and 10. Fig. 9 shows the impact of overlapping data on pseudo-labeling accuracy across different datasets. As shown in Fig. 9, the horizontal axis represents the number of overlapping data, and the vertical axis represents the pseudo-labeling accuracy. The results indicate that pseudo-labeling accuracy improves with the increasing in overlapping data and the number of labeled data involved in the transfer of labels. Fig. 10 illustrates the effect of overlapping data on detection accuracy for different datasets. As shown in

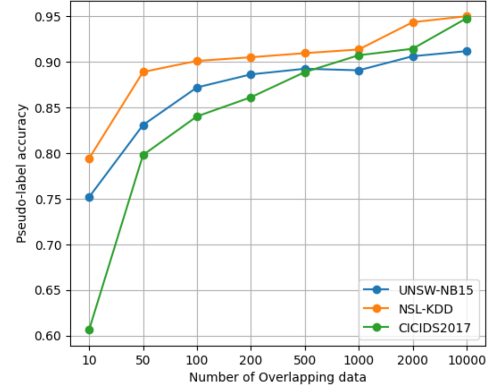


Fig. 9. Different numbers of overlapping data versus Pseudo-labeling accuracy on different datasets under MPdetector.

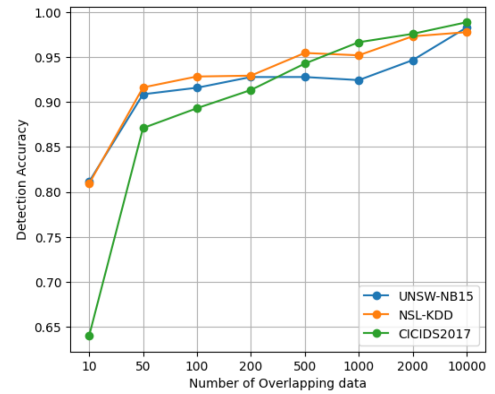


Fig. 10. Different numbers of overlapping data versus detection accuracy on different datasets under MPdetector.

Fig. 10, the horizontal axis represents the number of overlapping data, and the vertical axis represents detection accuracy. The results reveals that as the amount of overlapping data increases, the shared knowledge among clients grows, the effect of transfer learning is improved, and model detection performance is significantly improved. In summary, pseudo-labeling accuracy significantly impacts model performance. When the amount of overlapping data reaches 500, both pseudo-label accuracy and model detection performance achieve satisfactory results on relevant datasets. MPdetector significantly mitigates the adverse impact of limited overlapping data on model performance through its nonlinear data reconstruction method and label transfer approach. With the increasing of overlapping data, pseudo-labeling accuracy continues to rise, and detection accuracy is further improved.

E. Generalization Performance

This experiment uses the N-BaIoT dataset to evaluate whether MPdetector not only detects abnormal traffic within the training set but also enhances the ability to identify unknown abnormal traffic through knowledge transfer. Since the datasets of seven kinds of IoT devices in the N-BaIoT dataset contain two types of botnets, Gafgyt and Mirai, the data of these seven kinds of IoT devices are mainly used in this experiment, denoted

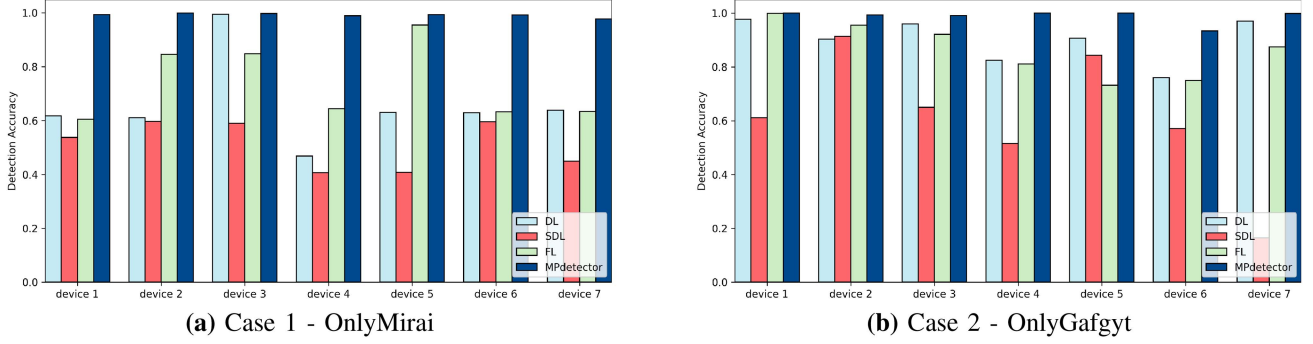


Fig. 11. Different kinds of IoT devices versus detection accuracy under different intrusion detection methods on two different cases.

as device1 - 7. The experiment involves two cases. Case 1 is called OnlyMirai, where Gafgyt attack data is removed from the training set and only Mirai attack data is retained to test the detection accuracy for Gafgyt attack data. Case 2 is called OnlyGafgyt, where the Mirai attack data is removed from the training set and only Gafgyt attack data is retained to test the detection accuracy for Mirai attack data.

The experimental results are shown in Fig. 11, where the horizontal axis represents the data of seven kinds of IoT devices and the vertical axis represents detection accuracy under different intrusion detection methods. The results show that the detection accuracy of MPdetector in identifying unknown abnormal traffic data is higher than that of other methods, which indicates that MPdetector has strong generalization ability. This is because MPdetector uses the similarity between labeled and unlabeled data for label transfer, makes full use of the information provided by unlabeled data, and then greatly improves the identification ability of unknown traffic data.

VI. RELATED WORK

With the development of machine learning and deep learning technology, intrusion detection methods based on deep learning have achieved good results in learning complex attack patterns and zero-day attack detection [10]. These algorithms have been used to detect anomalies in IoT networks, and have achieved good detection results. Abdelmoumin et al. [30] used hyper-parameter adjustment and ensemble learning to optimize the learning model to detect intrusion in the Internet of things in order to solve the problem of insufficient training data and single model easily falling into local optimum. Saba et al. [31] used genetic algorithm to select important features to improve the accuracy of the model, and machine learning algorithm to distinguish malicious traffic from regular traffic. Ge et al. [32] proposed an intrusion detection method based on deep learning network traffic classification, considering both multi-class and binary classification schemes. A feed-forward neural network with embedding layers and a second neural network for binary classification are used to leverage network embedding and transfer learning. However, IoT intrusion detection methods based on machine learning or deep learning rely on a large amount of data to achieve high performance. In reality, there is a phenomenon of data islands. Due to privacy and security concerns, data cannot

be shared between clients. Meanwhile, various types of client attack data are unbalanced. Different types of attack data from different clients also suffer from imbalance.

Federated learning offers a solution by allowing clients to collaboratively train a model without directly sharing local data, thus maintaining data security and achieving privacy protection. Idrissi et al. [33] proposed a federated learning based intrusion detection method to tackle privacy issues associated with centralized models. An autoencoder is used to calculate the intrusion score based on the reconstruction error, which achieves high accuracy and low false alarm rate. Li et al. [34] adopted federated learning with dynamic filtering and weighting strategies to protect data privacy and reduce communication overhead, and proved through experiments that the detection performance was improved with minimal network communication. Rashid et al. [35] show that federated learning can ensure privacy and security by having IoT clients only share parameter updates and a central server is responsible for aggregating and refining parameters. Karmakar et al. [36] enhanced the sharing of Autonomous Aerial Vehicle (AAV) data with federated learning, which enables multiple deep neural network models to collaboratively learn and update their parameters. Although the above federated learning schemes can solve the problem of privacy protection, they still require enough labeled data for local model training. In addition, data heterogeneity among participants can affect model performance. Consequently, numerous studies have proposed intrusion detection methods utilizing self-supervised learning and transfer learning approaches.

Self-supervised learning is an unsupervised approach that generates supervisory signals from data characteristics without manual labeling. Nguyen et al. [37] proposed a GNN-based self-supervised method for IoT intrusion detection, which learns node features through graph networks and enhances representation learning via traffic attribute prediction. Evaluations on real network traffic demonstrated its superiority in binary/multi-class attack detection, significantly outperforming GNN baselines in AUC and F1-score. Xu et al. [38] developed a graph contrastive learning framework using edge-attention encoders to capture traffic interaction patterns, achieving supervised-comparable accuracy with lower computational costs. However, these graph-based methods suffer from high complexity when handling large-scale node-edge relationships, making them unsuitable for IoT scenarios.

Adaptive transfer learning dynamically optimizes knowledge transfer between domains. Anley et al. [39] designed depth-adaptive CNNs with traffic-to-image conversion, enabling pre-trained vision models to process non-visual data and achieve cross-dataset feature transfer. Ajayi et al. [40] proposed an LSTM-based domain adaptation method for host intrusion detection, transferring knowledge via sequence modeling and fine-tuning to reduce label dependency. While effective, these approaches only support single-source-to-single-target transfer, requiring separate adaptations for multiple targets with substantial resource consumption.

Federated transfer learning seeks to transfer knowledge from diverse data sources to enhance local models' performance. This approach often outperforms models trained solely on local data [41]. Khoa et al. [17] introduced a collaborative learning framework that allows a target network, which has only unlabeled data, to learn efficiently from an original network with extensive labeled data. This framework improves performance by over 40% compared to existing deep learning methods. However, it is limited to interactions between two parties. When multiple participants are involved, updating models requires exchanging parameters in pairs, which complicates the process and increases both communication and computational costs. Cheng et al. [21] integrated federated 11 transfer learning with reinforcement learning to optimize intrusion detection in mobile edge computing. Their approach aims to achieve the highest detection accuracy with a limited number of clients while reducing costs. Ji et al. [20] proposed a method combining federated transfer learning with convolutional neural networks. This method builds a cloud model and then personalizes learning across organizations through transfer learning. After numerous iterations, it produces a highly effective detection model. Fan et al. [18] developed a federated transfer learning-based intrusion detection framework for 5 G IoT. The framework aggregates data through federated learning and customizes detection models through transfer learning, allowing IoT clients to share knowledge while preserving privacy, which in turn improves detection performance. Zhang et al. [8] proposed a lightweight anomaly based intrusion detection algorithm using a two-step fusion boosting strategy. This strategy evaluates data distribution variance with maximum average variance and adjusts model weights to enhance overall detection performance. However, these federated transfer learning methods typically require a central server to possess a large, labeled dataset to initialize the model, making them unsuitable for intrusion detection scenarios with extensive amounts of unlabeled data.

Unlike the above methods that require both extensive labeled traffic data for training initial detection models and homogeneous data distribution to achieve optimal performance, this paper propose MPdetector, a federated transfer learning approach for IoT intrusion detection, which supports the collaborative transfer learning of multiple clients and is effective even when most local data is unlabeled. It also addresses the challenges of IoT traffic data heterogeneity and the abundance of unlabeled data. Experimental results demonstrate that MPdetector can maintain superior detection accuracy even with limited labeled data and under significant feature heterogeneity conditions.

VII. CONCLUSION AND FUTURE WORK

To address the challenges of heterogeneous IoT network traffic data and the large volume of unlabeled data, this paper proposes a federated transfer learning method for IoT intrusion detection based on multi-party collaboration, called MPdetector. In MPdetector each client extracts feature representations of both label and unlabeled traffic data using an autoencoder. The task manager maps these feature representations to a unified feature space and applies a label transfer scheme to assign pseudo-labels to unlabeled data, enabling label transfer. Each client then can use a new proposed new mapping function to reconstruct the traffic data of other clients, achieving feature transfer of the traffic data. The effectiveness of MPdetector is verified by theoretical analysis and experiments. Experimental results show that MPdetector can achieve a detection accuracy as high as 98.2% and has strong generalization. Meanwhile the higher the number of labeled traffic data, the better the detection performance.

The effectiveness of MPdetector has been validated through experiments. The experimental results show that the intrusion detection accuracy of MPdetector is as high as 98.2%, which can effectively identify known and unknown abnormal traffic data, and has strong generalization. The higher the number of labeled traffic data, the better the detection performance.

MPdetector introduces a label transfer scheme to realize label transfer and reconstruct client traffic data to enable feature transfer across clients. However, the pseudo-labeling accuracy and the quality of data reconstruction may lead to irrelevant knowledge being learned, resulting in a decrease in detection accuracy. There are still some challenges to improve the pseudo-labeling accuracy and the quality of data reconstruction. In the next step, the optimized pseudo-label allocation method and data mapping method will be given to improve the pseudo-labeling accuracy and the quality of data reconstruction, and further improve the identification efficiency of traffic data. In addition, the current MPdetector approach involves multiple interactions between participating clients. In the future, the interaction between the task manager and the clients will be further optimized to ensure the detection accuracy while reducing the communication overhead of clients. In the future, it is expected to reduce the interaction between clients, and reduce the communication overhead of clients while ensuring the detection accuracy.

REFERENCES

- [1] L. Catarinucci et al., "An IoT-aware architecture for smart healthcare systems," *IEEE Internet Things J.*, vol. 2, no. 6, pp. 515–526, Dec. 2015.
- [2] Y. Tian, B. Zheng, and Z. Li, "Agricultural greenhouse environment monitoring system based on Internet of Things," in *Proc. 3rd IEEE Int. Conf. Comput. Commun.*, 2017, pp. 2981–2985.
- [3] H. Ghayvat, S. Mukhopadhyay, X. Gui, and N. Suryadevara, "WSN-and IoT-based smart homes and their extension to smart buildings," *Sensors*, vol. 15, no. 5, pp. 10350–10379, 2015.
- [4] F. Sattler, S. Wiedemann, K.-R. Müller, and W. Samek, "Robust and communication-efficient federated learning from non-i.i.d. data," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 31, no. 9, pp. 3400–3413, Sep. 2020.
- [5] M. Abomhara and G. M. Kjøen, "Cyber security and the Internet of Things: Vulnerabilities, threats, intruders and attacks," *J. Cyber Secur. Mobility*, vol. 4, pp. 65–88, 2015.

- [6] Y. R. Siwakoti, M. Bhurtel, D. B. Rawat, A. Oest, and R. Johnson, "Advances in IoT security: Vulnerabilities, enabled criminal services, attacks, countermeasures," *IEEE Internet Things J.*, vol. 10, no. 13, pp. 11224–11239, Jul. 2023.
- [7] NSFocuz Information Technology Co Ltd, "2020 cybersecurity insights," vol. 1, 2022. [Online]. Available: <https://nsfocuzglobal.com/wp-content/uploads/2021/06/2020-NSFOCUS-Cybersecurity-Insights.pdf>
- [8] J. Zhang, C. Luo, M. Carpenter, and G. Min, "Federated learning for distributed IIoT intrusion detection using transfer approaches," *IEEE Trans. Ind. Informat.*, vol. 19, no. 7, pp. 8159–8169, Jul. 2023.
- [9] A. L. Buczak and E. Guven, "A survey of data mining and machine learning methods for cyber security intrusion detection," *IEEE Commun. Surveys Tuts.*, vol. 18, no. 2, pp. 1153–1176, Second Quarter, 2016.
- [10] A. Aldweesh, A. Derhab, and A. Z. Emam, "Deep learning approaches for anomaly-based intrusion detection systems: A survey, taxonomy, and open issues," *Knowl.-Based Syst.*, vol. 189, 2020, Art. no. 105124.
- [11] H. Wang, J. Hou, and Z. Gong, "BotNet detection architecture based on heterogeneous multi-sensor information fusion," *J. Netw.*, vol. 6, no. 12, 2011, Art. no. 1655.
- [12] S. Hui et al., "Systematically quantifying IoT privacy leakage in mobile networks," *IEEE Internet Things J.*, vol. 8, no. 9, pp. 7115–7125, May 2021.
- [13] S. Agrawal et al., "Federated learning for intrusion detection system: Concepts, challenges and future directions," *Comput. Commun.*, vol. 195, pp. 346–361, 2022.
- [14] K. Chen et al., "Privacy preserving federated learning for full heterogeneity," *ISA Trans.*, vol. 141, pp. 73–83, 2023.
- [15] Y. Liu, Y. Kang, C. Xing, T. Chen, and Q. Yang, "A secure federated transfer learning framework," *IEEE Intell. Syst.*, vol. 35, no. 4, pp. 70–82, Jul./Aug. 2020.
- [16] J. Zhao, S. Shetty, and J. W. Pan, "Feature-based transfer learning for network security," in *Proc. 2017 IEEE Mil. Commun. Conf.*, 2017, pp. 17–22.
- [17] T. V. Khoa et al., "Deep transfer learning: A novel collaborative learning model for cyberattack detection systems in IoT networks," *IEEE Internet Things J.*, vol. 10, no. 10, pp. 8578–8589, May 2023.
- [18] Y. Fan, Y. Li, M. Zhan, H. Cui, and Y. Zhang, "IoTDefender: A federated transfer learning intrusion detection framework for 5G IoT," in *Proc. IEEE 14th Int. Conf. Big Data Sci. Eng.*, 2020, pp. 88–95.
- [19] Y. Otoum, V. Chamola, and A. Nayak, "Federated and transfer learning-empowered intrusion detection IoT applications," *IEEE Internet Things Mag.*, vol. 5, no. 3, pp. 50–54, Sep. 2022.
- [20] X. Ji, H. Zhang, and X. Ma, "A novel method of intrusion detection based on federated transfer learning and convolutional neural network," in *Proc. IEEE 10th Joint Int. Inf. Technol. Artif. Intell. Conf.*, 2022, pp. 338–343.
- [21] Y. Cheng, J. Lu, D. Niyato, B. Lyu, J. Kang, and S. Zhu, "Federated transfer learning with client selection for intrusion detection in mobile edge computing," *IEEE Commun. Lett.*, vol. 26, no. 3, pp. 552–556, Mar. 2022.
- [22] Y. Otoum, Y. Wan, and A. Nayak, "Federated transfer learning-based IDS for the internet of medical things (IoMT)," in *Proc. 2021 IEEE Globecom Workshops*, 2021, pp. 1–6.
- [23] S. Feng, B. Li, H. Yu, Y. Liu, and Q. Yang, "Semi-supervised federated heterogeneous transfer learning," *Knowl.-Based Syst.*, vol. 252, 2022, Art. no. 109384.
- [24] N. Moustafa and J. Slay, "UNSW-NB15: A comprehensive data set for network intrusion detection systems (UNSW-NB15 network data set)," in *Proc. 2015 Mil. Commun. Inf. Syst. Conf.*, 2015, pp. 1–6.
- [25] NSL-KDD dataset, 2009. Accessed: Feb. 08, 2021. [Online]. Available: <https://www.unb.ca/cic/datasets/nsf.html>
- [26] I. Sharafaldin et al., "Toward generating a new intrusion detection dataset and intrusion traffic characterization," in *Proc. 4th Int. Conf. Inf. Syst. Secur. Privacy*, 2018, pp. 108–116.
- [27] Y. Mirsky, T. Doitshman, Y. Elovici, and A. Shabtai, "Kitsune: An ensemble of autoencoders for online network intrusion detection," 2018, *arXiv:1802.09089*.
- [28] Y. Meidan et al., "N-BaIoT—Network-based detection of IoT BotNet attacks using deep autoencoders," *IEEE Pervasive Comput.*, vol. 17, no. 3, pp. 12–22, Third Quarter 2018.
- [29] A. Iscen, G. Tolias, Y. Avrithis, and O. Chum, "Label propagation for deep semi-supervised learning," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2019, pp. 5070–5079.
- [30] G. Abdelmoumin, D. B. Rawat, and A. Rahman, "On the performance of machine learning models for anomaly-based intelligent intrusion detection systems for the Internet of Things," *IEEE Internet Things J.*, vol. 9, no. 6, pp. 4280–4290, Mar. 2022.
- [31] T. Saba, T. Sadad, A. Rehman, Z. Mehmood, and Q. Javaid, "Intrusion detection system through advance machine learning for the Internet of Things networks," *IT Professional*, vol. 23, no. 2, pp. 58–64, 2021.
- [32] M. Ge, N. F. Syed, X. Fu, Z. Baig, and A. Robles-Kelly, "Towards a deep learning-driven intrusion detection approach for Internet of Things," *Comput. Netw.*, vol. 186, 2021, Art. no. 107784.
- [33] M. J. Idrissi et al., "Fed-ANIDS: Federated learning for anomaly-based network intrusion detection systems," *Expert Syst. Appl.*, vol. 234, 2023, Art. no. 121000.
- [34] J. Li, X. Tong, J. Liu, and L. Cheng, "An efficient federated learning system for network intrusion detection," *IEEE Syst. J.*, vol. 17, no. 2, pp. 2455–2464, Jun. 2023.
- [35] M. M. Rashid, S. U. Khan, F. Eusufzai, M. A. Redwan, S. R. Sabuj, and M. Elsharief, "A federated learning-based approach for improving intrusion detection in industrial Internet of Things networks," *Network*, vol. 3, no. 1, pp. 158–179, 2023.
- [36] R. Karmakar, G. Kaddoum, and O. Akhrif, "A novel federated learning-based smart power and 3D trajectory control for fairness optimization in secure UAV-assisted MEC services," *IEEE Trans. Mobile Comput.*, vol. 23, no. 5, pp. 4832–4848, May 2024.
- [37] H. Nguyen and R. Kashef, "TS-IDS: Traffic-aware self-supervised learning for IoT network intrusion detection," *Knowl.-Based Syst.*, vol. 279, 2023, Art. no. 110966.
- [38] R. Xu, G. Wu, W. Wang, X. Gao, A. He, and Z. Zhang, "Applying self-supervised learning to network intrusion detection for network flows with graph neural network," *Comput. Netw.*, vol. 248, 2024, Art. no. 110495.
- [39] M. B. Anley, A. Genovese, D. Agostinello, and V. Piuri, "Robust DDoS attack detection with adaptive transfer learning," *Comput. Secur.*, vol. 144, 2024, Art. no. 103962.
- [40] O. Ajayi and A. Gangopadhyay, "DAHID: Domain adaptive host-based intrusion detection," in *Proc. 2021 IEEE Int. Conf. Cyber Secur. Resilience*, 2021, pp. 467–472.
- [41] D. Gao, X. Yao, and Q. Yang, "A survey on heterogeneous federated learning," 2022, *arXiv:2210.04505*.



Li Lin (Member, IEEE) received the BS and MS degrees in mathematics from Guangxi Normal University, Guangxi Zhuang Autonomous Region, China, in 2001 and 2004, respectively, and the PhD degree in computer science from Beihang University, in 2009. She is currently an associate professor and a supervisor of master's candidates with the College of Computer Science, Beijing University of Technology, China. She has under taken research projects funded by National Natural Science Foundation of China and Beijing Natural Science Foundation and has been participating in various research projects supported by the National High-Tech Research and Development Program of China etc. Her current research interests include cloud computing and edge computing security, Big Data security and privacy protection, and artificial intelligence security.



ZhenKun Chen received the BS degree from the School of Information, Beijing Wuzi University, Beijing, China, in 2022, and the MS degree from the College of Computer Science, Beijing University of Technology, Beijing, China, in 2025. His research interests include federated transfer learning and intrusion detection technology. He has participated in some research projects funded by National Natural Science Foundation of China and Beijing Natural Science Foundation.